UNIVERSITY OF TECHNOLOGY SYDNEY

32998 - .NET Application Development

SPRING 2026

ASSIGNMENT-2 COVER SHEET

PROJECT TITLE	StudyConnect – a lab-aware study partner and assignment team finder (.NET 9 Blazor web application)
LEAD STUDENT NAME	Arya Kaslikar
LEAD STUDENT ID	26184921
GROUP MEMBER NAME(S) AND STUDENT ID(S)	Aditi Mangroliya – 26226264
SUBMISSION DATE	16 October 2026

AI USE RESTRICTIONS FOR PROGRAMMING WORK

Do not use AI to generate a substantial or complete solution to the programming assignment, or to bypass the learning outcomes of the assessment.

- AI must not be used to invent or misrepresent code execution, test results, screenshots, datasets, performance measurements, external libraries/APIs used, contributions of group members, or any other project detail.
- Permitted support may include concept explanations, syntax help, debugging suggestions, code review, test-case ideas, partial code generation, documentation/comments, and limited code examples where consistent with the assessment instructions.
- You must understand, review, run and test any AI-assisted code yourself, verify that it meets the specification, and be able to explain every submitted component and design decision.
- All AI use must be disclosed. AI output is not an authoritative source and does not replace reading subject materials, documentation, or instructions from teaching staff.

DECLARATION OF ORIGINALITY AND RESPONSIBILITY

☒ I confirm that the submitted work is the work of the named student(s), except where collaboration, source code, libraries, data, or AI assistance has been appropriately acknowledged. I understand the submitted program and can explain its implementation, testing and results.

Lead student signature:



_____ Date: 15/10/2026

Group member signature(s):



_____ Date: 15/10/2026

UNIVERSITY OF TECHNOLOGY SYDNEY

StudyConnect

Finding the right study partners and assignment teammates, from your own lab

1. Introduction and summary

StudyConnect is a web application that helps university students find study partners and, above all, form assignment teams with classmates from the same subject and the same lab. Anyone who has taken a group subject knows the problem it solves. In 32998, for example, the group assignment needs two or three students who all attend the same lab, yet most of us end up messaging classmates one by one, asking around in a busy lab, or joining whoever happens to be left. Students who are new, shy, studying part-time or simply less connected are the ones who lose out, and teams are often formed on convenience rather than on shared goals, compatible timetables and complementary skills. That mismatch is a common source of stress later in the semester.

StudyConnect turns this into a single, fair process. A student creates a profile with their interests and weekly free time, enrols in a subject and picks their lab, preferred team role, confidence level and target grade. The Team Finder then ranks every classmate in that lab who is still looking for a team, explains in plain language why each person is a good match, and suggests complete teams that can be created and invited in one click. Once a team exists, its members get a shared workspace with a live group chat, a team-health panel, a heat-map of common free time and a downloadable roster. The app also keeps the social features students already use: connection requests, real-time direct messages, study sessions, a notification bell and an Insights page with charts and machine-learning study communities.

It started as a Node.js, React and MongoDB prototype, and for this assignment we rebuilt it from the ground up in C# and .NET 9 so that it is faster, more secure, easier to test and a better fit for the subject. Our main goal goes beyond this assessment: we want to present StudyConnect at the university tech fest next semester and propose it to Dr Avinash Singh as a tool that could help 32998 students form balanced, same-lab teams in future offerings. This report explains what we built, how we built it, how the parts fit together and who did what.

2. Our main goal: tech fest and a proposal for the subject

We did not want to build something that would be marked and forgotten. Team formation is a real, recurring problem in every group-based subject, and StudyConnect already enforces the rules that matter in practice, such as teams of two to three and members from the same lab, instead of leaving them to be checked by hand. Our plan for the next semester is to polish the demo and collect feedback from classmates, present the project at the tech fest with a live demonstration of the Team Finder and the real-time team workspace, and propose to Dr Avinash Singh a small pilot in which one lab uses StudyConnect to form its assignment teams. Several design decisions in this version, such as the explainable match scores, the per-lab Insights charts and the automated test suite, were made with that audience in mind: a lecturer needs to trust how teams are suggested, and judges at a tech fest need to see that the product is reliable.

3. Development approach

We followed an iterative approach. The first version proved the idea; for this assignment we kept every original feature, rewrote the system in .NET, and then added the new Team Finder, Teams and Insights features in small, testable steps. We tracked our work in a shared GitHub repository, where each change is a commit and new features are developed on their own branch and merged into main through a pull request.

The solution has three projects with dependencies that only point inwards. StudyConnect.Core holds the entities, enums, service interfaces, the matching engine and the machine-learning code, and has no external packages, which keeps the algorithms simple to unit test. StudyConnect.Infrastructure implements the services with Entity Framework Core 9 and SQLite, sends e-mail, and runs an in-memory publish/subscribe hub for real-time updates. StudyConnect.Web is an ASP.NET Core Blazor Web App: the login, sign-up and password-reset pages are rendered statically so they can safely write the authentication cookie, and every other page is an interactive server component connected through SignalR.

Frameworks, libraries and tools: .NET 9 and C#; ASP.NET Core Blazor for the user interface; Entity Framework Core 9 with SQLite and LINQ for data access (the database file is created and seeded from JSON files on first run, so no database server is needed); SignalR, through Blazor, for live chat, notifications and toasts; NUnit for automated tests, including integration tests against an in-memory SQLite database; Visual Studio 2022 and Visual Studio Code with the .NET CLI; Git, GitHub and GitHub Actions for version control and continuous integration.

The subject's code requirements are met throughout the code base: more than two interfaces with dependency injection (for example ICompatibilityEngine, INotificationService and IMatchCriterion); polymorphism through the abstract MatchCriterion class and its six overriding subclasses, plus constructor and method overloading; generics such as Result<T> and a generic Combinations<T> iterator; LINQ with lambda expressions in every service; extension methods; and more than four distinct, responsive screens using buttons, dropdowns, sliders, modals, context menus, bar charts, lists and images.

The matching engine scores each pair of students as a weighted average of six criteria: same lab, shared interests, overlapping free time, complementary roles, goal alignment and skill balance. Each score comes with reasons a student can read. The TeamBuilder then tries combinations of the twelve best candidates and ranks whole teams by average pair compatibility, role coverage and common free time. For the machine-learning bonus, the Insights page turns every student's interests into TF-IDF vectors and groups them with our own K-Means implementation using k-means++ initialisation, discovering study communities and labelling each with its most important topics.

Security was treated as a requirement, not an extra: passwords are hashed with salted PBKDF2-SHA256, reset tokens are single-use, stored only as hashes and expire after 30 minutes, cookies are HTTP-only, forms use antiforgery tokens, and return URLs are checked to prevent open redirects. Quality is checked with 65 automated NUnit tests and a 101-case end-to-end test plan that walks a new student from sign-up to team formation and real-time chat.

4. Flowchart

Figure 1 shows how the main modules interact. The browser talks to the Blazor UI; pages only depend on service interfaces, and the concrete services are supplied through dependency injection. Services read and write data through Entity Framework Core, call the matching, machine-learning and security code in the Core project, and publish events to the real-time notifier, which pushes live updates back to the navigation bar, the notification bell and any open page.

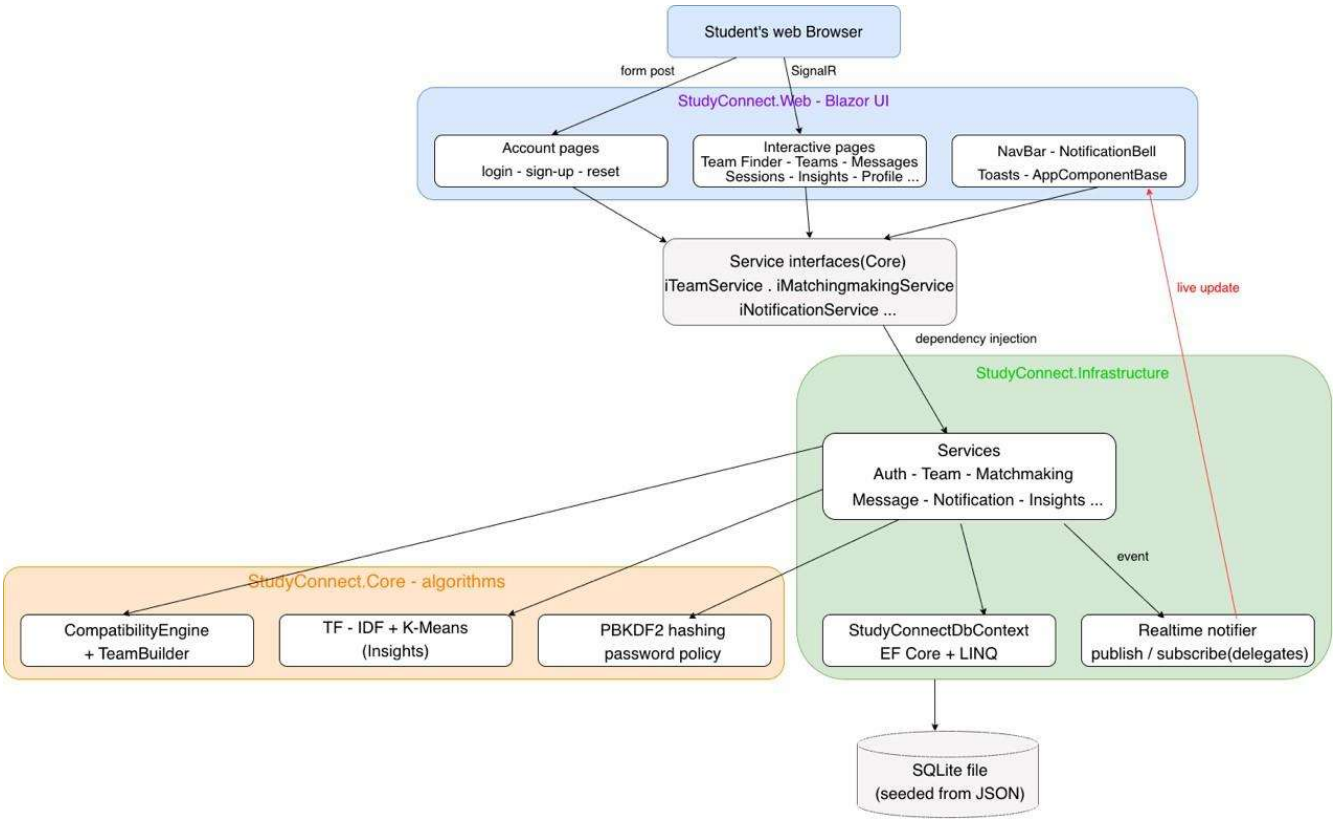


Figure 1. Component diagram: how the Web, Core and Infrastructure modules interact.

Figure 2 follows the core journey through the Team Finder, from choosing a subject to a team invitation arriving in a classmate's notification bell.

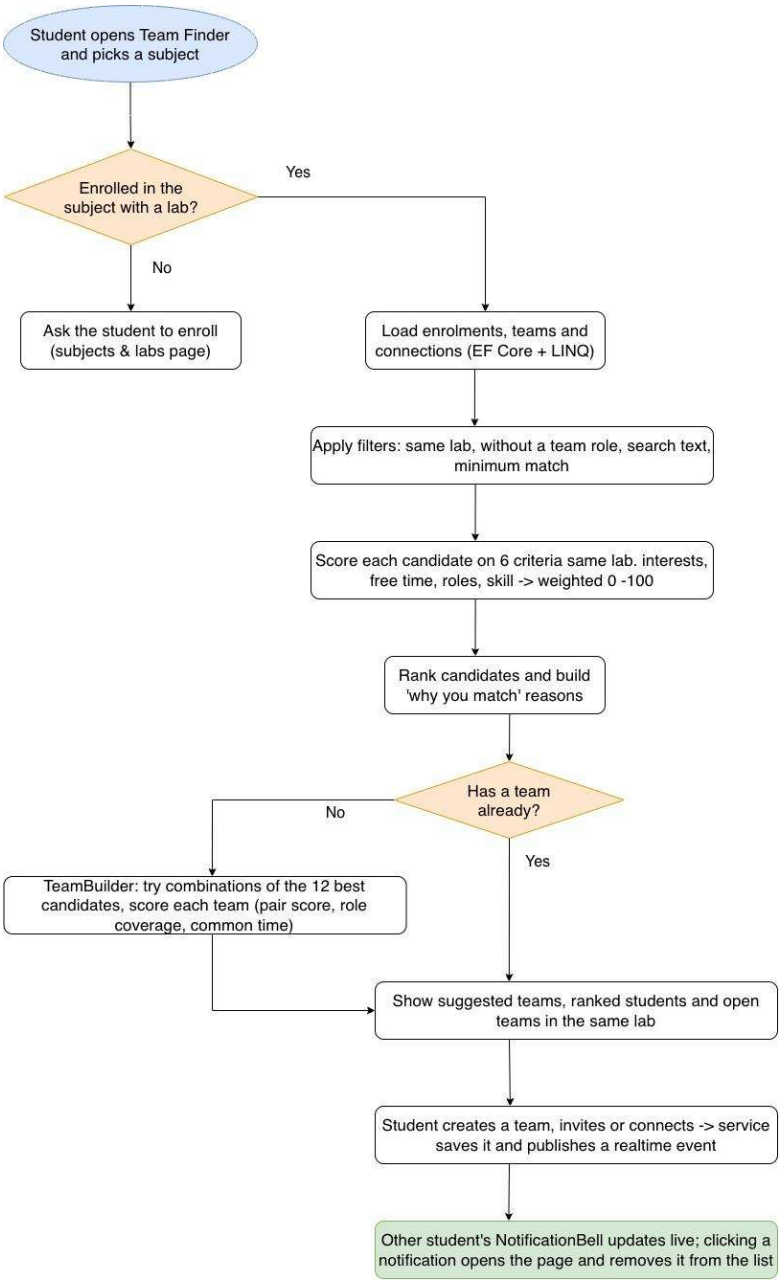


Figure 2. Flowchart of the Team Finder and team-invitation process.

5. Role of team members

We are a team of two and have worked on StudyConnect together for a while. We built the original Node.js and React prototype together, planned the .NET rewrite together, and agreed the feature list for this assignment as a pair. Within that shared work, each of us took clear ownership of different parts.

Arya Kaslikar (team lead) was responsible for the overall architecture and the bulk of the .NET implementation. He designed the three-layer solution structure and the Entity Framework Core data model, and wrote the JSON seeding that creates the 30 demo students. He implemented authentication and security (PBKDF2 hashing, password reset, cookie handling), the matching engine with its six criteria, the TeamBuilder, the K-Means and TF-IDF code behind the Insights page, real-time messaging and team chat, and most of the Blazor pages. He also wrote the original NUnit test suite, set up the GitHub repository and the GitHub Actions build-and-test workflow, maintained the README and run instructions, cleaned up the code before submission, and is responsible for registering the project and submitting the final zip. As team lead he also planned the final week of work, reviewed and merged pull requests, and kept the main branch in a working state when changes from both of us had to be combined.

Aditi Mangroliya owned the notification improvements, quality checking and the documentation for this submission. Before this change, a notification stayed in the bell after a student had clicked it and acted on it, so the list filled up and the unread badge stayed misleading. Aditi added a DismissAsync method to the notification service interface and implementation, which deletes the notification only if it belongs to the signed-in student, so one student can never remove another student's notifications. She then changed the NotificationBell component so that clicking a notification removes it, refreshes the badge count and opens the related page, and added an NUnit test that checks both the deletion and the ownership rule. She developed this on a separate feature branch, using commits and a pull request so the leader's main branch stayed untouched until review. When the GitHub Actions build failed on one commit, she found and fixed the cause, and she tested the finished feature locally with two browser sessions.

Aditi also checked the project against the marking guide, confirmed which bonus-mark criteria the project already meets (Blazor instead of Windows Forms, Entity Framework with LINQ, and a machine-learning algorithm), and reviewed the matching and machine-learning logic. That review found that the TF-IDF model is used only on the Insights page and not in the Team Finder score, and she designed an improvement that is described in the future-work section. She also wrote this report and prepared the AI use statement.

Some work was shared. We both took part in planning and design discussions, kept in regular contact while working on separate parts, tested the application using our end-to-end test plan, and reviewed each other's changes before they reached the main branch. Working on separate branches meant we could make progress at the same time without overwriting each other's work.

The table below summarises each member's main responsibilities.

Member	Role	Main responsibilities in this submission
Arya Kaslikar	Team lead, architecture and core development	Solution architecture; EF Core data model and seeding; authentication and security; matching engine, TeamBuilder, TF-IDF and K-Means; real-time messaging and team chat; most Blazor pages; original NUnit suite; GitHub repository, CI workflow and README; final submission

Aditi Mangroliya	Feature development, quality and documentation	Notification dismissal (service method, bell UI, and pull request; local and end-to-end testing; marking-guide and bonus-criteria check; matching and ML review and improvement design; project report and AI use statement
Both	Shared	Joint development including the original prototype; planning and design; testing with the 101-case test plan; code review; tech fest and proposal plan

Table 1. Summary of team member roles.

6. Use of AI tools

In line with the subject's rules, all AI use is disclosed in the attached AI Use Statement. In summary, Aditi used Claude (Anthropic) to suggest and partially generate code for the notification-dismissal feature, generate NUnit test-case ideas, improve code comments, review the code and the matching and machine-learning logic, help debug a build error, and help draft and structure this report. Arya used GitHub Copilot extensively for code completion, including the matchmaking code, and ChatGPT (GPT-5.6) to help structure the report, generate the more complex machine-learning code, create NUnit test cases and dummy demo data, and write accurate code comments and to help implement secure authentication. GitHub Copilot was also involved in one merge commit. Every AI-assisted change was reviewed, run and tested by us, and we can explain all of the submitted code.

7. Future work towards the tech fest

Before the tech fest we plan to use the TF-IDF model inside the Team Finder score, so that rare shared interests count for more than very common ones. We have already evaluated this idea on the demo data. We also plan to show unread chat messages in the notification bell, add a show/hide button to password fields, import enrolments and lab lists from the timetable, collect peer feedback after each assignment, and eventually learn the criterion weights from how well past teams worked together. These features would make StudyConnect ready for a pilot with a real lab.

8. How to run and use StudyConnect

1. In Visual Studio 2022 (with .NET 9), open StudyConnect.sln, set StudyConnect.Web as the start-up project and press F5. In Visual Studio Code, install the .NET 9 SDK and the C# Dev Kit extension, open the solution folder, then run “dotnet run --project src/StudyConnect.Web” in the terminal. Either way, the app opens at http://localhost:5080. No database server is needed: a SQLite file is created in App_Data and filled with 30 demo students on the first run. Full steps are also in the ReadMe.txt file in the submission.
2. Log in as alice@uts.edu.au with the password Demo@1234 (all demo accounts use this password), or create a new account with Create an account.
3. Open Team Finder, choose 31927 and look at the Suggested teams, Students and Open teams tabs. Expand Why this score? on a student card to see the six matching criteria, and use the filters and the minimum-match slider.

4. Click Create team & invite on a suggestion. In a private browser window, log in as bob@uts.edu.au, open My Teams and accept the invitation. Both windows update live, and the team workspace offers a group chat, a team-health panel and a CSV roster export.
5. Explore Messages, Connections, Sessions and Insights. Click the bell to see notifications; clicking one opens the related page and removes it from the list.
6. To run the automated tests, open Test → Test Explorer and click Run All, or run dotnet test in the solution folder.

References

Arthur, D. & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms, 1027–1035.

Salton, G. & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

Microsoft. (2025). ASP.NET Core Blazor documentation. https://learn.microsoft.com/aspnet/core/blazor

Microsoft. (2025). Entity Framework Core documentation. https://learn.microsoft.com/ef/core

NUnit Project. (2025). NUnit documentation. https://docs.nunit.org Diagrams.

Draw.io https://app.diagrams.net/

Appendix A. Application screenshots

Screenshots of the running application with the seeded demo data, logged in as alice@uts.edu.au.

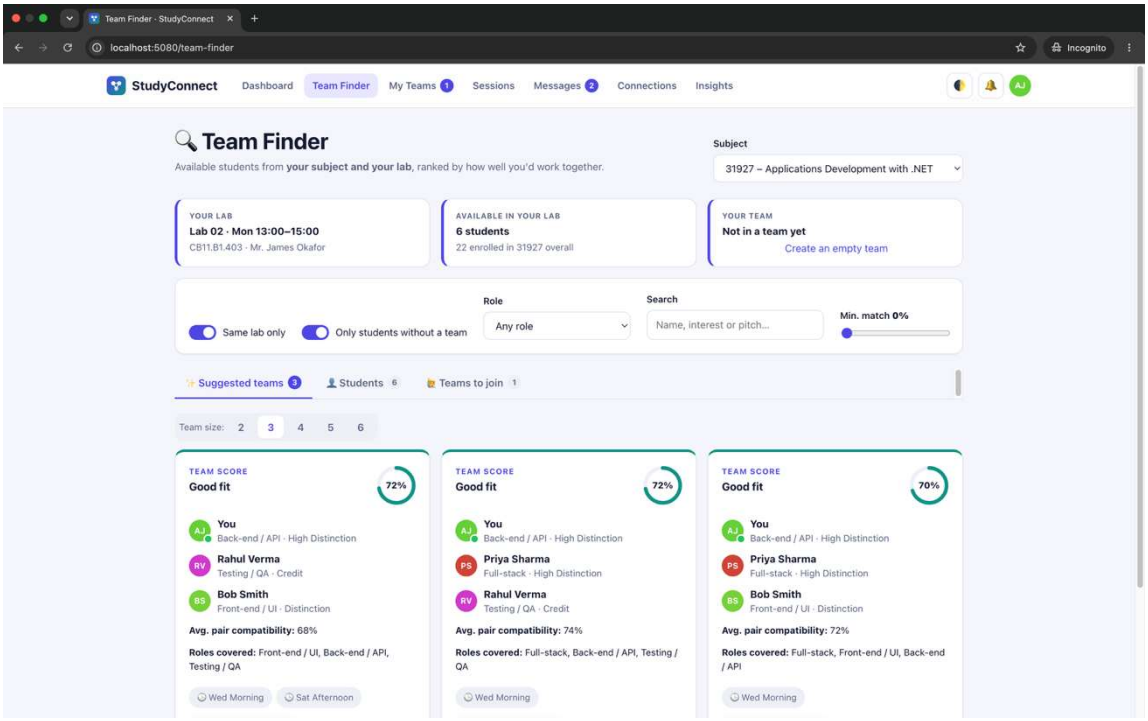


Figure 3. Team Finder: suggested teams for 31927 Lab 02, with team score, roles covered and common free time.

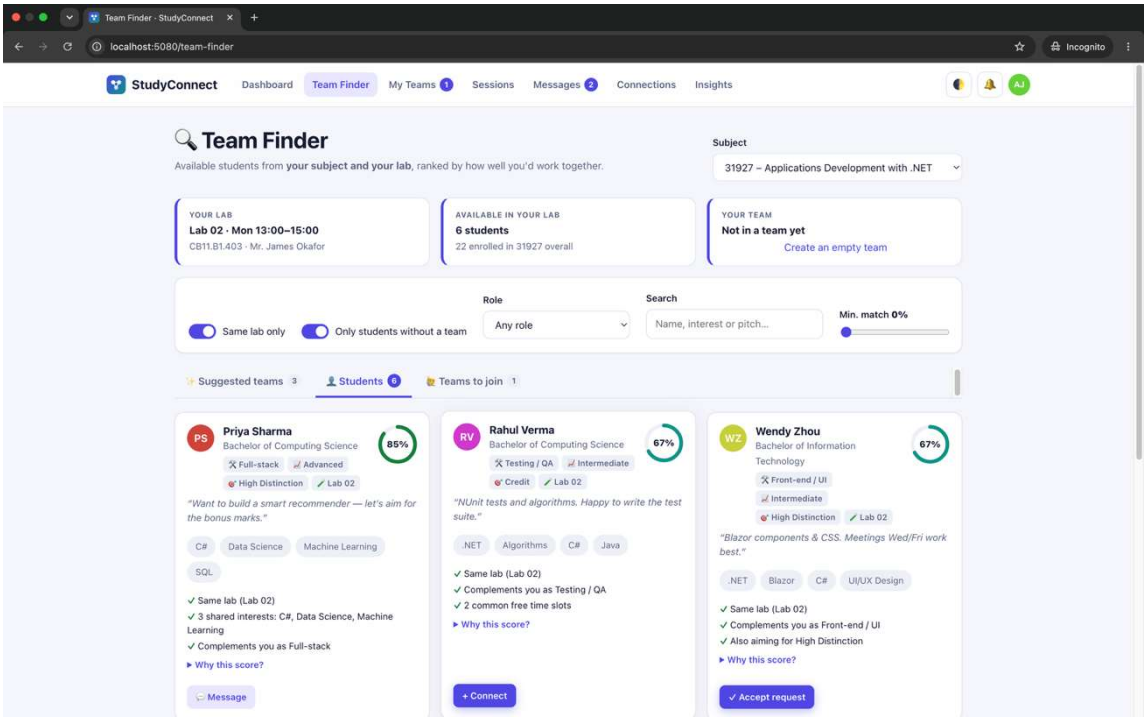


Figure 4. Team Finder: students from the same lab ranked by compatibility, with the reasons behind each score.

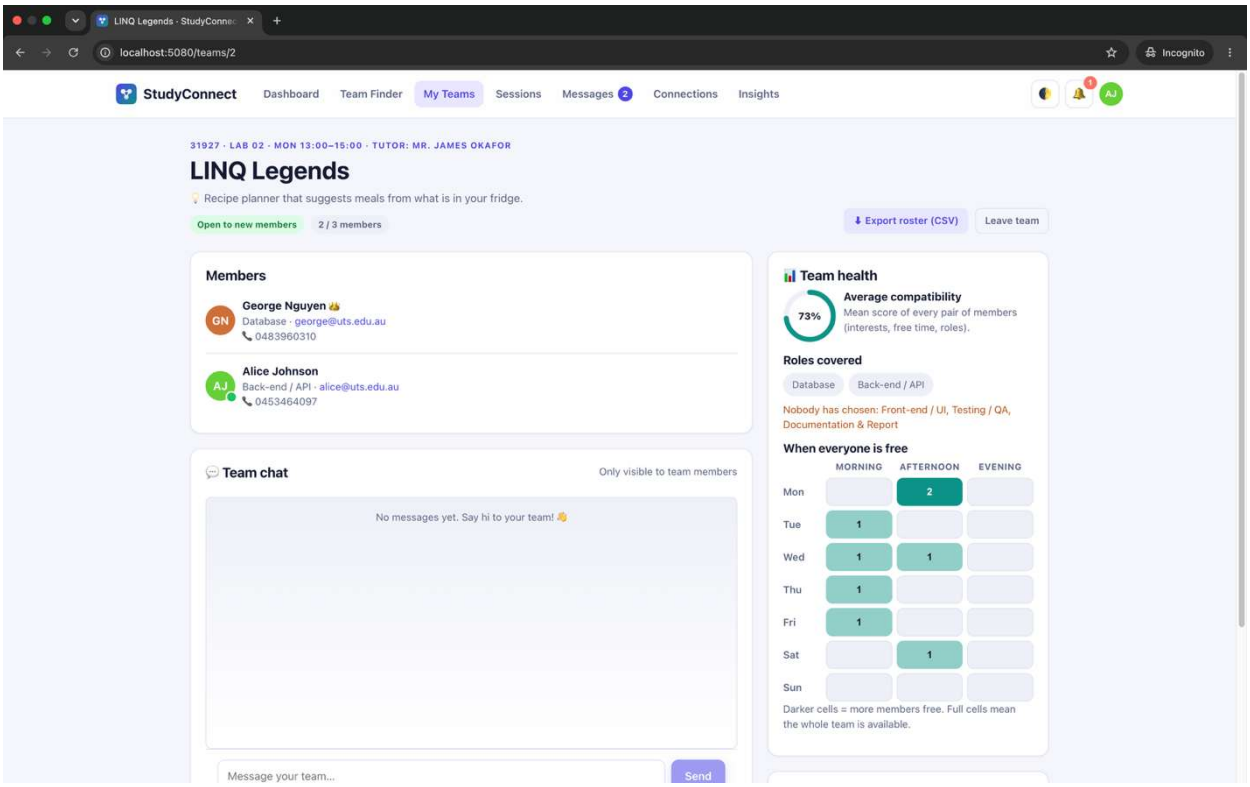


Figure 5. Team workspace: members, live team chat, team-health panel and free-time heat-map.

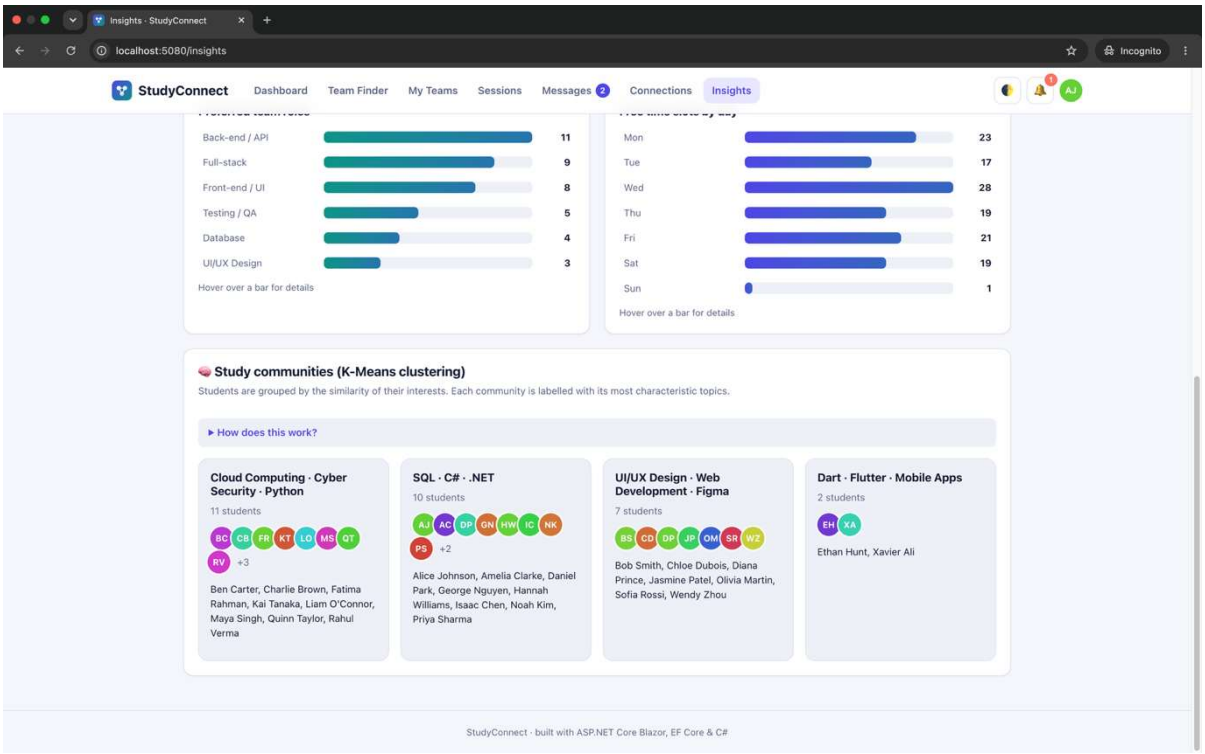


Figure 6. Insights: study communities found with TF-IDF and K-Means clustering.

UNIVERSITY OF TECHNOLOGY SYDNEY

AI USE STATEMENT — ASSIGNMENT-2

Attach this completed statement as an appendix to your submission. Disclose all AI-enabled tools used, including uses that did not produce code retained in the final submission.

Student name(s) and ID(s)	Arya Kaslikar – 26184921; Aditi Mangroliya – 26226264
Subject code	32998
Project / assessment title	StudyConnect – Assignment 2

☒ I did use any AI-enabled tool for this assessment. (If selected, proceed to the declaration.)

1. AI tools and model names

List every AI-enabled tool used (for example ChatGPT, GitHub Copilot, Microsoft Copilot, Gemini, Claude, Grammarly or other coding assistants). Record the model name shown in the tool, or "not shown".

AI tool or product	Model name (as shown)	Date(s) used
Claude (Anthropic) – used by Aditi	Claude Opus 5.5	13–15 Oct 2026
GitHub Copilot – used by Arya for code completion across the project (including the matchmaking code) and one merge commit (Copilot CLI)	Not shown	08-15 Oct 2026
ChatGPT (OpenAI) – used by Arya	GPT-5.6	12 Oct 2026

2. What the AI tools were used for (tick all that apply)

☒ Explained programming concepts or APIs	☒ Suggested algorithms, approaches or pseudocode
☒ Helped with syntax or small code examples	☒ Generated or completed code
☒ Debugged errors or explained error messages	☒ Reviewed/refactored code I had written
☒ Suggested unit tests, edge cases or test data	☒ Generated documentation, comments or README text
☒ Helped with data analysis or visualisation code	☒ Assisted with version-control / build / tooling commands
☒ Proofread or revised report text	☒ Other: helped draft and structure the report text from our notes

3. Representative examples of AI use

Give 2–3 examples. Summarise what you asked the tool to do and explain what you accepted, changed, tested or rejected. Do not paste full chat logs unless requested.

Tool / model	Prompt or task summary	How I used, changed or rejected the output	Where it affected the submission
Claude (Opus 5.5)	Remove a notification from the bell once it is clicked.	Used the suggested service method and bell change as a starting point and applied them ourselves on a feature branch; fixed a build error CI found; tested in two browsers.	IServices.cs, NotificationService.cs, NotificationBell.razor
Claude (Opus 5.5)	Suggest NUnit tests for the dismissal feature.	Kept one test for owner-only deletion and the unread count; full suite passed (65).	ServiceTests.cs
Claude (Opus 5.5)	Review matching/ML logic; improve comments.	TF-IDF matching idea evaluated but not merged (future work); comments edited by us.	Report section 7; comments in changed files

Tool / model	Prompt or task summary	How I used, changed or rejected the output	submission
GitHub Copilot; ChatGPT (GPT-5.6)	Code completion for the matchmaking engine; generating the more complex ML code (TF-IDF, K-Means), NUnit test cases and dummy demo data; accurate code comments; report structure.	Reviewed the suggestions before committing; adapted them to our project structure and removed unnecessary generated code, comments and files in clean-up commits (13 Oct). Tested the matching and ML code with NUnit tests (CompatibilityEngine, TeamBuilder, K-Means and TF-IDF tests) and checked the demo data loads and the GitHub Actions build passes.	Matching and MachineLearning code, NUnit tests, Data/Seed JSON files, code comments, report structure

4. Verification and validation

☒ Ran and tested all submitted code and reviewed outputs, errors and edge cases	☐ Checked AI-suggested APIs/libraries against official documentation or course materials
☒ Confirmed that reported outputs, screenshots and performance results came from my/our actual program execution	☒ Reviewed AI-assisted code for correctness, security, originality, licensing/attribution and assessment requirements
☒ I/we can explain every submitted code component and justify the implementation and testing decisions	☒ No confidential, personal or unauthorised third-party material was uploaded to an AI tool

Briefly describe one specific check you performed and any change made as a result:

We ran dotnet test after the change (65 passed) and checked in two browser windows that clicking a notification opens the right page, removes it from the list and lowers the badge.

GitHub Actions reported a failed build on one commit; we traced it to a missing semicolon in our edit of NotificationBell.razor and fixed it before opening the pull request.

Declaration

I/we confirm that this statement accurately describes all AI-enabled tools used for this assessment and the extent to which their outputs influenced or appear in the submission. I/we followed the assessment instructions, independently reviewed and tested the submitted work, and take responsibility for the final code, report and results. I/we understand that prompts, outputs, commit history or other supporting evidence may be requested for an academic integrity review.



Lead student signature: _____ Date: 15/10/2026



Group member signature(s): _____ Date: 15/10/2026